

1. Цели и задачи работы

Цель настоящей работы — комплексное исследование мобильных операционных систем и мобильных приложений с позиции их применения в корпоративной среде, прежде всего в организациях атомной отрасли России.

Для достижения цели поставлены следующие **задачи**:

- рассмотреть архитектуру и характеристики ведущих мобильных операционных систем — Android, iOS, а также российских разработок;
- проанализировать особенности применения мобильных ОС в корпоративной среде: модели развёртывания, управление устройствами, требования к безопасности;
- изучить специфику внедрения мобильных технологий в атомной отрасли: цифровые обходы, интеграция с АСУ ТП, требования регуляторов;
- классифицировать виды мобильных приложений и сравнить подходы к их разработке;
- выявить особенности разработки и эксплуатации корпоративных приложений с акцентом на отечественные инструменты.

Актуальность темы обусловлена ускоренной цифровизацией производственных процессов в условиях импортозамещения и необходимостью обеспечения информационной безопасности критической информационной инфраструктуры (КИИ)¹.

2. Рынок мобильных ОС

2.1. Глобальный рынок

По итогам 2024 года мировой рынок мобильных операционных систем характеризуется явной двухполюсной структурой. Android занимает около 72–78 % рынка смартфонов, iOS — около 27–22 % в зависимости от региона. Прочие платформы суммарно не превышают 1 %.

¹КИИ — критическая информационная инфраструктура; понятие закреплено Федеральным законом № 187-ФЗ от 26.07.2017.

Таблица 2.1 — Распределение глобального рынка мобильных ОС (2024 г.)

Операционная система	Доля рынка (глобально)	Основной производитель
Android	72–78 %	Google / AOSP-сообщество
iOS	22–27 %	Apple Inc.
Прочие	< 1 %	KaiOS, HarmonyOS и др.

В корпоративном сегменте соотношение смещается в сторону iOS в странах Западной Европы и Северной Америки, тогда как в России корпоративный рынок исторически ориентирован на Android-устройства и сейчас целенаправленно переходит на отечественные платформы.

3. Android: архитектура и особенности

Android построен на многоуровневой архитектуре поверх ядра Linux. Уровни, от нижнего к верхнему:

1. **Ядро Linux** (Linux Kernel) — управление аппаратными ресурсами, драйверами, безопасностью на уровне процессов.
2. **Аппаратный уровень абстракции** (HAL, Hardware Abstraction Layer) — унифицированный интерфейс для взаимодействия системы с конкретным железом.
3. **Android Runtime** (ART) — виртуальная машина с AOT-компиляцией байт-кода Dalvik; заменила Dalvik VM начиная с Android 5.0.
4. **Нативные библиотеки C/C++** (libc, OpenGL ES, SQLite, WebKit и др.).
5. **Java API Framework** — набор API, предоставляемых приложениям: менеджеры активностей, пакетов, уведомлений, телефонии и т.д.
6. **Системные приложения** — стандартный набор (звонки, SMS, браузер, настройки).

Ключевая особенность Android — открытость исходного кода (AOSP, Android Open Source Project), что позволяет создавать производные платформы без лицензионных отчислений Google. Именно на AOSP базируются российские ОС KVADRA_OS и РЕД ОС М.

Существенный недостаток Android-экосистемы — **фрагментация**: по данным на 2024 год, одновременно в эксплуатации находятся устройства под управлением версий от Android 10 до Android 14, что усложняет поддержку корпоративных приложений и своевременное получение патчей безопасности.

4. iOS: архитектура и особенности

iOS (с 2019 г. iPadOS для планшетов) построена на основе Darwin — Unix-подобного ядра XNU (гибрид Mach и BSD). Архитектура iOS также многоуровневая:

1. **Core OS** — ядро Darwin, криптографические примитивы, Bluetooth, Wi-Fi.
2. **Core Services** — фундаментальные сервисы: iCloud, Core Data, Core Location, Core Motion.
3. **Media** — AVFoundation, Core Image, Core Audio, Metal (GPU API).
4. **Cocoa Touch** — UIKit, MapKit, GameKit; уровень, с которым непосредственно взаимодействуют приложения.

В отличие от Android, iOS — **закрыва́тая** система: установка приложений возможна только через App Store (или корпоративный MDM-профиль). Это повышает безопасность, но ограничивает гибкость. Каждое приложение выполняется в изолированной **sandbox**-среде с жёсткими ограничениями на доступ к системным ресурсам. Обновления ОС выпускаются централизованно Apple и распространяются синхронно на все поддерживаемые устройства, что практически исключает проблему фрагментации.

5. Российские мобильные операционные системы

На отечественном рынке выделяются две группы мобильных ОС: производные AOSP и дистрибутивы на базе Linux.

Таблица 5.1 — Российские мобильные операционные системы

ОС	Основа	Разработчик	Целевой сегмент
KVADRA_OS	AOSP (Android)	НТЦ ИТ РОСА / КНТТ	Планшеты, корп. устройства
РЕД ОС М	AOSP (Android)	РЕД СОФТ	Корпоративный, госсектор
Аврора ОС	Sailfish OS / Linux	Открытая мобильная платформа	Гос. и корп. сектор, КИИ
ROSA Mobile	Linux (Mer/Nemo)	РОСА	Исследовательский

KVADRA_OS и **РЕД ОС М** сохраняют совместимость с Android-приложениями (APK), что облегчает миграцию. Однако Google Mobile Services (GMS) в них недоступны, что требует переработки приложений, зависящих от сервисов Google.

Аврора ОС принципиально отличается: её ядро построено на Linux без использования AOSP, а API несовместим с Android. Платформа развивается «Открытой мобильной платформой» (ОМП) при поддержке «Ростелекома» и ориентирована на применение в государственных структурах и на объектах КИИ, где использование иностранных платформ ограничено.

6. Корпоративная среда: модели развёртывания

В корпоративной мобильности применяются три основные модели управления устройствами, отличающиеся балансом между гибкостью для сотрудника и контролем со стороны организации.

Таблица 6.1 — Модели развёртывания корпоративных мобильных устройств

Модель	Расшифровка	Владелец устройства	Применение
BYOD	Bring Your Own Device	Сотрудник	Офисный персонал, частичная занятость
COPE	Corporate-Owned, Personally Enabled	Организация	Линейный персонал с личными нуждами
COBO	Corporate-Owned, Business Only	Организация	КИИ, производство, режимные объекты

Для управления корпоративными устройствами применяются системы **MDM** (Mobile Device Management) и более широкие **EMM** (Enterprise Mobility Management), включающие **MAM** (управление приложениями) и **MIM** (управление информацией). На объектах КИИ, в том числе в атомной отрасли, доминирует модель **COBO**, поскольку она обеспечивает полный контроль над программной средой устройства.

7. Аврора ОС в корпоративной среде

Аврора ОС обладает архитектурными особенностями, выгодно отличающими её от Android в контексте КИИ. Ключевые компоненты:

- **Ядро Linux** — стабильная и аудируемая основа без фирменных слоёв AOSP.
- **Wayland Compositor** (Lipstick) — замена X11/SurfaceFlinger; управляет отрисовкой пользовательского интерфейса.

- **Sailfish Silica / Qt** — UI-фреймворк на базе QML и Qt; собственный, не связанный с Android API.
- **D-Bus** — межпроцессное взаимодействие системных демонов.
- **MDM-агент** — встроенный агент управления, позволяющий централизованно конфигурировать, блокировать и обновлять устройства.

Средства управления Аврора ОС объединены в **Аврора Центр** — корпоративную консоль MDM. Она обеспечивает инвентаризацию устройств, дистанционную установку приложений, политики паролей и шифрования, а также удалённое уничтожение данных (remote wipe).

Важным преимуществом является сертификация ФСТЭК² России, что позволяет использовать Аврора ОС для обработки сведений ограниченного доступа.

8. Мобильные технологии в атомной отрасли

8.1. Цифровые обходы оборудования

Одним из наиболее востребованных приложений мобильных технологий на предприятиях Росатома являются **системы цифровых обходов** (инспекционные маршруты). Традиционно обходы выполнялись с бумажными чек-листами, что порождало задержки в передаче данных и риск ошибок при ручном вводе.

Цифровой обход на планшете с Аврора ОС позволяет:

- сканировать QR-коды или NFC-метки на единицах оборудования для автоматической идентификации;
- вносить показания приборов и выявленные замечания непосредственно в мобильное приложение;
- прикреплять фотографии дефектов;
- синхронизировать данные с корпоративными системами (EAM, ERP) после выхода из зоны с ограниченным радиодоступом.

8.2. Интеграция с АСУ ТП

Интеграция мобильных решений с АСУ ТП³ реализуется через специализированные шлюзы и OPC UA-серверы, изолирующие технологический уровень от корпоративного. Мобильные устройства никогда не подключаются напрямую к SCADA-системам; взаимодействие осуществляется по схеме:

²ФСТЭК — Федеральная служба по техническому и экспортному контролю; орган, уполномоченный выдавать сертификаты соответствия требованиям защиты информации.

³АСУ ТП — автоматизированная система управления технологическим процессом.

Передаваемые данные — как правило, телеметрия для отображения (только чтение); команды управления с мобильных устройств исключены из архитектуры безопасности.

9. Требования безопасности КИИ и сетевая сегментация

Объекты атомной отрасли относятся к объектам КИИ первой категории значимости. Ключевые регуляторные требования:

Таблица 9.1 — Регуляторные требования к мобильным решениям на объектах КИИ

Документ / регулятор	Основное требование для мобильных устройств
ФЗ № 187-ФЗ (КИИ)	Использование ПО и оборудования из реестра российской продукции
Приказ ФСТЭК № 239	Сегментация сети, контроль съёмных носителей, мониторинг инцидентов
НП-026-16 (Росатом)	Запрет несанкционированных подключений к АСУ ТП
Приказ ФСБ № 378	Шифрование каналов связи сертифицированными СКЗИ

Сетевая сегментация (Network Segmentation) является фундаментальным принципом защиты. Сеть предприятия делится как минимум на три уровня:

1. **Корпоративная сеть** (уровень L3) — доступ с мобильных устройств через Wi-Fi или мобильный Интернет с VPN; применяется MDM-контроль.
2. **DMZ** (демилитаризованная зона) — граничный сегмент; здесь размещаются шлюзы, обеспечивающие однонаправленную передачу данных из промышленной сети.
3. **Промышленная (технологическая) сеть** (уровень L1/L2) — физически изолирована; мобильные устройства в неё не включаются.

Для мобильных устройств на объектах Росатома обязательно применение:

- сертифицированных СКЗИ⁴ (например, ViPNet) для VPN;

⁴СКЗИ — средства криптографической защиты информации.

- MDM-политик, запрещающих установку стороннего ПО, использование Bluetooth и личных облачных сервисов;
- Аврора ОС как сертифицированной платформы вместо Android или iOS.

10. Виды мобильных приложений

Мобильные приложения классифицируются по архитектурному подходу к разработке. Каждый подход предполагает различный баланс между производительностью, стоимостью разработки и охватом платформ.

Таблица 10.1 — Сравнение видов мобильных приложений

Вид	Исполнение	Доступ к API ОС	Доставка
Нативное	Машинный код / ART	Полный	App Store / MDM
Кросс-платформенное	Нативное (компиляция) или JIT (интерпретация)	Полный / частичный	App Store / MDM
Гибридное	WebView	Через JS-мост	App Store / MDM
PWA	Браузер	Ограниченный	URL (без магазина)

11. Нативная разработка

Нативные приложения создаются с использованием официальных SDK конкретной платформы и компилируются в машинный код (или байт-код, исполняемый нативной средой платформы).

Таблица 11.1 — Языки и инструментарий нативной разработки

Платформа	Язык	Инструментарий
Android	Kotlin (основной), Java	Android Studio, Gradle
iOS	Swift (основной), Obj-C	Xcode, SwiftUI / UIKit
Аврора ОС	C++ / Qt, Python	Qt Creator, Аврора SDK

Нативный подход обеспечивает максимальную производительность и полный доступ ко всем возможностям платформы: камере, NFC, Bluetooth, датчикам. Это критически важно для промышленных приложений, работаю-

щих с периферийным оборудованием. Основной недостаток — необходимость поддерживать отдельную кодовую базу для каждой платформы.

Для Аврора ОС использование Qt/C++ дополнительно выгодно тем, что Qt имеет давнюю историю применения во встраиваемых и промышленных системах, что упрощает интеграцию с существующим ПО предприятий.

12. Кросс-платформенная разработка

Кросс-платформенные фреймворки позволяют использовать единую (или почти единую) кодовую базу для нескольких платформ.

Таблица 12.1 — Сравнение кросс-платформенных фреймворков

Фреймворк	Язык	Рендеринг UI	Поддержка Авроры
Flutter	Dart	Собственный движок (Skia/Impeller)	Экспериментальная
React Native	JavaScript / TypeScript	Нативные компоненты ОС	Отсутствует
KMP / CMP	Kotlin	Нативный (expect/actual) или Compose	Да (через QtBindings)

Flutter компилирует Dart-код в нативный ARM-код и рисует UI через собственный графический движок, не используя виджеты платформы. Это обеспечивает визуальную согласованность между платформами, но усложняет интеграцию с системным UI.

React Native использует JavaScript-мост для вызова нативных компонентов, что создаёт накладные расходы на коммуникацию между JS-поток и нативным потоком.

Kotlin Multiplatform (KMP) — принципиально иной подход: только **бизнес-логика** (сеть, база данных, доменный слой) является общей, тогда как UI разрабатывается нативно для каждой платформы. **Compose Multiplatform (CMP)** расширяет KMP, добавляя возможность разделять и UI-код на базе Jetpack Compose.

13. Аврора ОС и Kotlin Multiplatform

Аврора Мультиплатформа - инициатива «Открытой мобильной платформы» по интеграции KMP с Аврора ОС. Цель - позволить командам разра-

батывать мобильные приложения для Android, iOS и Аврора ОС с максимально общей кодовой базой.

Таблица 13.1 — Распределение кода в проекте КМР с поддержкой Аврора ОС

Слой приложения	Реализация в КМР + Аврора
Сетевой (HTTP, REST)	Общий Kotlin-код (Ktor)
Локальная БД	Общий Kotlin-код (SQLDelight)
Бизнес-логика / ViewModel	Общий Kotlin-код
UI — Android	Jetpack Compose (Kotlin)
UI — iOS	SwiftUI / UIKit (Swift)
UI — Аврора ОС	Qt/QML (C++) через КМР-биндинги

Архитектурно биндинг работает следующим образом: КМР-библиотека компилируется в нативный бинарный файл (.so) для платформы Аврора (aarch64-linux); Qt-приложение подключает эту библиотеку и вызывает её функции через С-интерфейс. Таким образом, бизнес-логика пишется один раз на Kotlin, а UI остаётся нативным для каждой ОС.

Это решение является стратегически важным для атомной отрасли: оно позволяет параллельно поддерживать приложения для Android-устройств (общего назначения, административный персонал) и для Аврора ОС (режимные объекты, операционный персонал), не дублируя значительную часть кода.

14. Заключение

В ходе работы достигнута поставленная цель и решены все задачи.

- Архитектура мобильных ОС.** Рассмотрены многоуровневые архитектуры Android и iOS, проанализированы отечественные платформы: KVADRA_OS и РЕД ОС М (производные AOSP) и Аврора ОС (Linux-платформа). Показано, что для объектов КИИ Аврора ОС является единственной сертифицированной отечественной платформой.
- Корпоративная среда.** Сравнение моделей BYOD / COPE / COBO показало, что для промышленных предприятий и режимных объектов обоснована модель COBO с применением MDM. Аврора ОС обеспечивает встроенные инструменты управления через Аврора Центр.
- Атомная отрасль.** Определены основные сценарии применения: цифровые обходы оборудования, сбор телеметрии, работа с технической документацией. Установлено, что интеграция с АСУ ТП допу-

стима только через однонаправленные шлюзы в DMZ. Требования ФСТЭК и Росатома фактически предписывают использование Авроры ОС.

4. **Классификация приложений.** Систематизированы нативный, кросс-платформенный, гибридный подходы и PWA; для промышленных приложений приоритетен нативный подход.
5. **Корпоративные приложения и Аврора Мультиплатформа.** КМР позволяет разделять бизнес-логику между Android и Аврора ОС, снижая затраты на разработку при сохранении нативного UI для каждой платформы. Это стратегически перспективное направление для цифровизации Росатома в условиях импортозамещения.

Практическая значимость результатов состоит в возможности их применения при обосновании выбора мобильной платформы и стека разработки для корпоративных проектов предприятий с высокими требованиями к информационной безопасности.